

RESULTS

Exam Results

744 / 1000

PASSED

45 of 60 correct (75%) · Pass mark: 720

Domain Breakdown

D1

Agentic Architecture & Orchestration

10/14 (71%) +193pts

D2

Tool Design & MCP Integration

7/11 (64%) +115pts

D3

Claude Code Configuration & Workflows

9/12 (75%) +150pts

D4

Prompt Engineering & Structured Output

9/12 (75%) +150pts

D5

Context Management & Reliability

10/11 (91%) +136pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes  $0.80 \times 0.27 \times 1000 = 216$  points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60

CORRECT 45

INCORRECT 15

SKIPPED 0

Q01 · D4 · 4.4 VALIDATION-RETRY / PYDANTIC-VALIDATION · q-4-4-009 · Structured Data Extraction Pipeline

CORRECT

Your extraction pipeline uses tool\_use with a JSON schema, so outputs always parse. Finance still reports invoices where line items do not sum to the stated total, and where the due date precedes the invoice date. The team wants retries to receive precise, per-field error messages. What should you add?

D

A more capable model, since semantic errors indicate the current one is under-powered

No model tier guarantees semantic correctness; the fix is validation logic around the model, and an upgrade is the maximum-cost non-answer.

**A** Tighter JSON schema constraints: numeric minimum and maximum bounds on every amount field and format checks on every date field

Range and format constraints check fields in isolation; both reported failures are relationships between fields, which a JSON schema cannot express.

**B** An instruction telling the model to double-check its own extraction in the same request before responding

Same-session self-review retains the generation context and its bias, and produces no machine-readable error for the retry loop to feed back.

**C** A Pydantic model whose validators encode the sum and date-order rules, feeding its validation errors into retries

Cross-field semantic rules live in validation code; Pydantic validators express them and raise per-field errors the retry request can quote verbatim.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Pydantic as the validation layer)

Q02 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-002 · Multi-Agent Research System **INCORRECT**

A multi-agent research system has a coordinator that spawns a web search subagent and a document analysis subagent sequentially across separate API turns. The web search completes in 8 seconds and the document analysis completes in 12 seconds, giving a total latency of 20 seconds. The two subagents are investigating independent topics and do not depend on each other's results. How should the architect reduce this latency?

**C** Allow the web search subagent to directly invoke the document analysis subagent after it finishes, removing the coordinator round-trip

Direct subagent communication violates the hub-and-spoke architecture where all communication flows through the coordinator. This also would not achieve parallel execution — it would still be sequential.

**D** Use `fork_session` to split the coordinator into two parallel branches, one for each subagent

`fork_session` is for divergent exploration of the same problem from a shared baseline, not for parallel subagent invocation. Emitting multiple Task tool calls in a single coordinator response is the correct mechanism for spawning concurrent subagents. (your answer)

**A** Merge the web search and document analysis into a single subagent to reduce coordination overhead

Merging agents violates the principle of specialised subagents with scoped tool access. It also does not reduce total processing time — the merged agent still needs to do both tasks, and loses the benefit of parallel execution.

**B** Have the coordinator emit both Task tool calls in a single response to spawn both subagents in parallel, reducing total latency to roughly 12 seconds

When subagents investigate independent topics, the coordinator can emit multiple Task tool calls in a single response. Both subagents run concurrently, so total latency is determined by the slower one (12 seconds) rather than the sum (20 seconds). **(correct answer)**

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

Q03 · D4 · 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS · q-4-3-007 · Structured Data Extraction Pipeline **INCORRECT**

An extraction pipeline uses 'tool\_use' with a 15-required-field schema. For some document types, only 8 of those fields ever appear in the source material. The model consistently fabricates plausible values for the missing 7, and the instruction 'do not fabricate data' has not helped. Should the team (A) keep all fields required and add post-extraction validation, or (B) redesign the schema? Which is correct and why?

**B** Redesign the schema to make the 7 sometimes-absent fields optional/nullable, allowing the model to return null instead of fabricating

Making fields optional/nullable when source data may be absent removes the schema-level pressure to fabricate. The model can legitimately return null for missing information, which is a correct representation of reality. This addresses the root cause at the schema level. **(correct answer)**

**C** Create separate schemas for each document type, each containing only the fields present in that document type

Multiple schemas add maintenance complexity and require document type classification before extraction. Making fields nullable in a single schema is simpler and equally effective. **(your answer)**

**D** Switch to prompt-based JSON extraction where you can instruct the model to omit absent fields entirely

Prompt-based JSON sacrifices schema compliance guarantees. tool\_use with optional fields gives both flexibility for absent data and schema validation for present data.

**A** Keep all fields required and add post-extraction validation to detect and remove fabricated values

Post-extraction validation cannot reliably distinguish fabricated values from correct ones — fabricated values are designed to look plausible. The schema itself forces fabrication by making all fields required.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable fields)

Lesson 4.2: Structured Output with Tool Use (Schema does not prevent fabrication)

Q04 · D2 · 2.5 BUILT-IN-TOOLS / EDIT-RECOVERY · q-2-5-002 CORRECT

A developer is using Claude Code to update a configuration value in a file. They use the Edit tool, but it fails because the text they are trying to match appears in three places in the file. What is the correct next step?

**A** Switch to using Bash with sed to perform the replacement, as it handles multiple matches natively.

Sed bypasses the built-in tool designed for this task and may introduce errors with special characters. The exam expects candidates to use the correct built-in tool sequence.

**B** Use Grep to find which of the three occurrences is the correct one, then provide more surrounding context to Edit to make the match unique.

When Edit fails due to non-unique matches, the first recovery step is to include more surrounding context so the match becomes unique. Grep helps identify the right occurrence and its context. This keeps you within the Edit workflow.

**C** Fall back to Read to load the full file, then Write the complete modified file.

Read + Write is the fallback when Edit cannot be made to work at all. The correct first step is to provide more surrounding context to make the Edit match unique. Read + Write should only be used after that approach fails.

**D** Use Glob to find all files containing the configuration value, then edit each one individually.

Glob searches file paths, not file contents. It cannot find files containing a specific value. Additionally, the problem is within a single file, not across multiple files.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Edit tool)

Claude Code: Built-in Tools ›

Q05 · D4 · 4.4 VALIDATION-RETRY / RETRY-BOUNDARY · q-4-4-008 · Structured Data Extraction Pipeline CORRECT

A validation-retry loop still fails on the same documents after three attempts each. In which situations will further retries not help? (Select 2)

**D** The information exists only in an external document that is not provided in context.

Retries cannot reach data outside the model's context; the fix is supplying the external document, not another attempt.

**C** The validation service timed out on the last attempt.

A timeout is a transient failure, which is precisely the class of error a retry is likely to resolve.

**A** The model placed a correct value in the wrong field on the last attempt.

A value in the wrong field is a semantic validation error; retry-with-error-feedback naming the misplaced field guides the next attempt to correct it.

**B** The required information is simply absent from the source document.

No amount of retrying makes missing information appear; the guide marks absent data as the boundary of retry effectiveness.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Q06 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-START q-1-7-004 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

INCORRECT

A Claude Code agent has been working on a feature branch for 45 minutes and has accumulated extensive context about the codebase. The developer notices that several tool results from early in the session (file reads from 40 minutes ago) are now stale because a colleague pushed changes to those files. The agent is making recommendations based on the outdated file contents. What is the best recovery strategy?

**B** Start a completely new session with no context from the previous session

Starting fresh discards 45 minutes of accumulated context and architectural understanding. The agent would need to rediscover all the insights it already found about the codebase.

**C** Start a fresh session with a summary of the key findings and decisions, then read the changed files for current state.

Fresh start with summary injection is the prescribed recovery strategy for stale tool results. It preserves the valuable insights and decisions from the previous session while eliminating the stale data. Reading the changed files in the new session ensures the agent works from current state. (correct answer)

**A** Continue in the current session and simply ask the agent to re-read the files a colleague changed, so it picks up the latest contents.

Re-reading updates the file contents, but the stale results remain in the context window. The agent may still reference the outdated content from earlier in the conversation, leading to contradictory reasoning. (your answer)

**D** Use fork\_session to create a new branch that excludes the stale tool results

fork\_session creates a branch from the current state, which includes the stale tool results. It does not allow selective exclusion of context. It is designed for divergent exploration, not for stale context recovery.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.7: Session State and Resumption (Decision matrix)

Q07 · D3 3.1 CLAUDE-ND-HIERARCHY / ENFORCEMENT q-3-1-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

Project-level ``.claude/CLAUDE.md`` says 'use 4-space indentation matching the existing codebase.' A senior architect has 'use 2-space indentation' in their user-level ``.claude/CLAUDE.md``. In recent sessions the architect's code has come back in 2 spaces and broken the build. The team needs a guarantee that 4-space indentation is applied on every save. What should they do?

**A** Add a `PostToolUse` hook that runs the team's formatter after every `Write/Edit`, so 4-space indentation is enforced regardless of what Claude generates

Anthropic's memory docs are explicit that `CLAUDE.md` is delivered as a user message with 'no guarantee of strict compliance,' and that 'if two rules contradict each other, Claude may pick one arbitrarily.' The docs themselves point at hooks for this case: hooks 'execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.' A `PostToolUse` hook running the formatter is the only option here that gives a hard guarantee.

**B** Leave the rule in project-level ``.claude/CLAUDE.md`` — the more specific scope wins on conflicts, so the project rule will override the architect's user-level preference

This is the popular paraphrase, but it's not what Anthropic's docs say. The docs describe a load order (broadest scope to most specific, so project instructions appear in context after user instructions) but explicitly state files are 'concatenated into context rather than overriding each other' and conflicts 'may [be] pick[ed] arbitrarily.' The team has already seen that assumption fail in production.

**C** Ask the architect to delete their user-level ``.claude/CLAUDE.md`` so there is no conflict to resolve

This treats a personal config file as a team problem and doesn't scale — every new teammate would need to police their own home directory, and any future contradiction (from a different file, an `@import`, or a ``.claude/rules/`` entry) would resurface the same fragility. The fix is to remove reliance on guidance-style config for a rule that must be enforced, not to remove the conflicting file.

**D** Move the 4-space rule into a ``.CLAUDE.local.md`` at the project root so it is appended last and reads after the user-level file

Load order does append ``.CLAUDE.local.md`` after ``.CLAUDE.md`` within a directory, but the docs are careful to call this 'load order,' not precedence, and warn that conflicts may resolve arbitrarily. ``.CLAUDE.local.md`` is also gitignored, so a team standard cannot live there.

WHERE THIS COMES FROM

Lesson 3.1: `CLAUDE.md` Hierarchy and Scoping (Loading order and conflict handling)

Anthropic — How Claude remembers your project (memory docs) ↗

Q08 · D2 · 2.4 MCP-SERVER-INTEGRATION / CONFIG-SCOPE · q-2-4-008

CORRECT

The data platform team needs to share their Snowflake and PostgreSQL MCP server configurations with all team members whilst allowing individual developers to experiment with a personal API integration server. Where should each configuration be placed?

**D** Place the database servers in environment variables and reference them in both `.mcp.json` and ``.claude.json``.

Environment variables are used for credentials (connection strings, API keys), not for MCP server configuration. The server definitions themselves belong in `.mcp.json` or `~/claude.json` depending on scope.

**C** Place all three servers in `~/claude.json` and have each developer copy the configuration manually.

User-level configuration is not version-controlled and requires manual synchronisation. Team-wide servers belong in project-level `.mcp.json` for consistency and change tracking.

**A** Place all three servers in the project-level `.mcp.json` so they are version-controlled and consistent across the team.

Personal experimental servers should not be in project-level configuration. Changes to personal servers would create unnecessary merge conflicts and force experimental integrations on the entire team.

**B** Place the Snowflake and PostgreSQL servers in the project-level `.mcp.json`, and personal API integration servers in user-level `~/claude.json`.

Project-level `.mcp.json` is version-controlled and shared — correct for team-wide database integrations. User-level `~/claude.json` is machine-specific — correct for personal experimental servers that should not affect other team members.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Scoping hierarchy)

Claude Code: MCP Server Configuration ↗

Q09 · D5 · 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING · q-5-6-004 · Multi-Agent Research System · CORRECT

A multi-agent research system finds conflicting data on a pharmaceutical compound's efficacy. A 2022 peer-reviewed clinical trial reports 78% efficacy, while a 2024 preprint reports 45% efficacy with a larger sample size. The synthesis agent must produce a recommendation for a medical advisory board. Which approach correctly handles the conflicting sources?

**C** Present both findings with full provenance: source identity, publication date, peer-review status, sample size, methodology, and any methodological differences that may explain the discrepancy. Let the advisory board weigh the evidence

For a medical advisory board, preserving full provenance is critical. The board needs to weigh peer-review status, sample size, methodology differences, and temporal context. Annotating both sources with complete metadata enables informed human judgment on conflicting scientific evidence.

**D** Exclude both findings and report that no reliable consensus exists, recommending further research before any conclusions

Excluding available evidence is unhelpful. The advisory board can make informed decisions with properly attributed conflicting data. Withholding findings because they conflict reduces the board's ability to assess the situation.

**B** Average the two values (61.5%) and present it as the best estimate with a note about variance between studies

Averaging conflicting scientific results is methodologically invalid. The studies may have different populations, methodologies, or endpoints. The average has no scientific meaning and obscures the actual disagreement.

**A** Use the 2024 preprint as it has a larger sample size and is more recent, noting it supersedes the 2022 study

Recency and sample size alone do not determine reliability. The preprint has not undergone peer review, and simply picking one source discards valuable context that the advisory board needs for informed decision-making.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflicts intact)

Q10 · D3 3.2 SLASH-COMMANDS-SKILLS / SKILLS-PLACEMENT q-3-2-014 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

The team creates a reusable `/extract-service` skill that guides Claude Code through the standard steps of extracting a module from the monolith into a microservice (identify boundaries, create service scaffold, migrate code, update call sites, add tests). Where should this skill be stored so that every team member who clones the repository has access?

**B** In the repository's `.claude/skills/` directory, committed to version control

`.claude/skills/` is project-scoped and version-controlled. When any team member clones the repository, the skill is automatically available as a `/extract-service` command. Updates to the skill flow through normal git pull, ensuring all developers have the same version.

**C** In the repository-root `CLAUDE.md` as an inline procedure

`CLAUDE.md` is for always-loaded standards, not on-demand workflows. An extraction procedure is invoked when needed, not applied to every session. Placing it in `CLAUDE.md` wastes tokens when developers are not extracting services.

**A** In `~/.claude/skills/` on each developer's machine, distributed via the team wiki

`~/.claude/skills/` is user-scoped and not version-controlled. Distributing via wiki requires manual copying and creates drift between developers' versions. The requirement is automatic availability on clone.

**D** In a shared MCP server that all team members connect to

MCP servers provide tool integrations and external data access, not reusable prompt-based workflows. A skill file is the correct mechanism for a guided multi-step procedure.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Project skill scoping)

Claude Code: Skills ↗

Q11 · D2 2.1 TOOL-SCHEMA-DESIGN / SPLITTING q-2-1-008

INCORRECT



A developer productivity platform has a tool called `analyze_content` with the description 'Analyses content from various sources.' Logs show the agent uses it for web scraping, document parsing, and code analysis indiscriminately, leading to poor results for each use case. What is the most effective fix?

**B** Improve the tool description to list all supported content types and their expected formats.

A better description on a generic tool does not change the fact that the tool's processing logic is too broad. When a tool serves multiple distinct purposes, the fix is to split it into purpose-specific tools. **(your answer)**

**C** Split `analyze_content` into purpose-specific tools: `extract_web_results`, `parse_document`, and `analyze_code`, each with a narrow remit.

Tool splitting replaces a generic tool with purpose-specific alternatives that each have a clear, narrow description and defined input/output contracts. Each resulting tool handles one use case well, which eliminates ambiguity about when to use it. **(correct answer)**

**A** Add detailed few-shot examples to the system prompt showing which types of content the tool handles well and which it does not.

Few-shot examples add token overhead and treat the symptom. The root cause is that a single generic tool is trying to serve three distinct use cases, each requiring different processing logic.

**D** Implement a routing layer that inspects the input content type and dispatches to the appropriate internal processing function within `analyze_content`.

An internal routing layer hides the content type distinction from the agent. Splitting into separate tools exposes the distinction through tool descriptions, enabling the LLM's natural language understanding to select the right tool.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool splitting)

Lesson 2.1: Tool Interface Design (Tool descriptions)

Q12 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-016 · Multi-Agent Research System **CORRECT**

A coordinator delegates to a data collection agent (`'web_search'`, `'database_query'`) and an analysis agent (`'calculate'`, `'chart_generate'`). In testing, the analysis agent calls `'web_search'` directly to fetch extra context, bypassing the data collection agent. What is the architectural violation and how should it be fixed?

**A** The analysis agent's `allowedTools` includes `web_search`; remove it so only the data collection agent has it.

Each agent should only have tools relevant to its specific role. The analysis agent should have `calculate` and `chart_generate`, not `web_search`. If the analysis agent needs additional data, it should request it through the coordinator, which then delegates to the data collection agent. This enforces the scoped tool access principle.

**B** Add a system prompt instruction telling the analysis agent not to use `web_search` even though it has access

Prompt instructions are probabilistic. The agent is already occasionally using `web_search` despite the architectural intent. The fix is to remove the tool from `allowedTools`, providing deterministic enforcement rather than hoping the prompt works.

**D** Implement a `PostToolUse` hook that logs when the analysis agent uses `web_search` for monitoring purposes

Logging the violation does not prevent it. The architectural fix is to remove `web_search` from the analysis agent's `allowedTools`, not to monitor its misuse.

**C** Allow the analysis agent to keep `web_search` access since it sometimes needs additional context for better analysis

This violates scoped tool access and bypasses the data collection agent's role. If the analysis agent needs more data, the correct flow is: analysis agent signals the coordinator, coordinator delegates to the data collection agent, results flow back through the coordinator.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access)

Anthropic: Claude Agent SDK Overview ↗

Q13 · D3 · 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-FORMATTING · q-3-1-016 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

The team wants to ensure that every Java file written by Claude Code during refactoring is automatically formatted with the project's Checkstyle rules before being saved. Developers occasionally forget to run the formatter manually. What is the correct hook configuration?

**A** A `PreToolUse` hook on the `Write` tool that runs Checkstyle on the content before the file is written

`PreToolUse` fires before the tool executes. The file has not been written yet, so there is no file on disk to format. `PreToolUse` is suited for blocking or validating inputs, not for post-processing outputs.

**B** A `PostToolUse` hook on file write operations that runs the Checkstyle formatter on the written file, automatically correcting any style violations

`PostToolUse` fires after the file is written to disk. A hook that runs the Checkstyle formatter on the output file ensures every written Java file conforms to the project's style rules, regardless of what the model generated. This provides deterministic enforcement without relying on prompt instructions.

**D** A `PreToolUse` hook on the `Read` tool that verifies all Java files are Checkstyle-compliant before they are read into context

Checking files on read does not address the requirement. The goal is to format files when they are written, not to validate files when they are read. Existing legacy files may not be Checkstyle-compliant and should not block reading.

**C** Add 'Always run Checkstyle before saving files' to the `CLAUDE.md` system prompt

Prompt instructions are probabilistic. The question states developers 'occasionally forget', and an LLM-based instruction has the same failure mode. A hook provides deterministic enforcement that cannot be skipped.

WHERE THIS COMES FROM

Claude Code: Hooks ›

Anthropic: Agent SDK Hooks ›

Q14 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / STRUCTURED-METADATA q-1-3-001 · Multi-Agent Research System

INCORRECT

A synthesis agent produces a report with several claims that have no source attribution. The web search and document analysis subagents are working correctly and returning well-sourced results. What is the most likely root cause?

C The web search subagent needs to return results in a different format

The web search subagent is working correctly. The issue is how the coordinator passes its results to the synthesis agent.

D The synthesis agent needs access to the web search tool directly

Giving the synthesis agent web search tools violates the principle of scoped tool access and does not address the context passing failure.

B The coordinator passes content to the synthesis agent stripped of structured metadata like source URLs and document names.

Context passing must include structured data that separates content from metadata. Without source URLs and document names, the synthesis agent has no attribution information to include. (correct answer)

A The synthesis agent's system prompt does not instruct it to include citations, so it drops source attribution even when given well-sourced input.

Even with a citation instruction, the synthesis agent cannot attribute claims to sources that were never passed to it. The lesson is explicit: do not blame the synthesis prompt; the root cause is the coordinator dropping structured metadata. (your answer)

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q15 · D3 3.6 CICD-INTEGRATION / WORKTREE-COORDINATION q-3-6-009 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

INCORRECT

Two Claude Code instances are running in separate git worktrees: Instance A is extracting the payment service (branch: extract/payment) and Instance B is extracting the inventory service (branch: extract/inventory). Both instances need to modify the shared OrderService.java file — Instance A to remove payment logic and Instance B to remove inventory logic. How should the team coordinate this?

**D** Use file locking via git to prevent both instances from modifying OrderService.java simultaneously

Git does not have built-in file locking for standard workflows. While Git LFS supports file locking, it is designed for binary files, not source code coordination. Sequential merge-and-rebase is the standard approach for coordinating parallel branches.

**C** Create a third Claude Code instance dedicated to modifying only shared files, with Instances A and B handling everything else

A dedicated shared-file instance adds coordination complexity without clear benefit. It needs to understand both extraction contexts to make correct modifications. Sequential merging with rebasing is simpler and more reliable.

**A** Let both instances modify OrderService.java independently on their branches and resolve the merge conflict when merging to main

Allowing both instances to make independent, potentially contradictory modifications to the same file risks complex merge conflicts that are difficult to resolve correctly. Coordinating changes to shared files upfront is safer. (your answer)

**B** Have Instance A complete and merge first, then Instance B rebases onto the updated main before modifying OrderService.java

Sequencing modifications to shared files avoids merge conflicts. Instance A completes its changes to OrderService.java and merges. Instance B then rebases onto the updated main, seeing Instance A's changes, and makes its modifications with full awareness of the current state. This is the standard coordination pattern for parallel Claude Code instances modifying shared files. (correct answer)

WHERE THIS COMES FROM

Anthropic: Claude Code Documentation ↗

Git: Worktrees ↗

Q16 · D5 · 5.6 INFORMATION-PROVENANCE / STRUCTURED-PROVENANCE · q-5-6-008 ·

Technical Documentation Maintenance System

INCORRECT

A docs team has Claude Code generate architecture guides by synthesising source code, inline comments, commit messages, and ADRs. After several edits, the guides no longer indicate which statements came from which source. A developer later questions whether a specific architectural constraint in a guide is still valid. What approach preserves source attribution?

**A** Instruct Claude Code to add footnotes to the generated documentation citing the original source for each statement

Prose footnotes are fragile — they are lost or degraded during subsequent editing passes, summarisation, or reformatting. The team has already experienced this problem through 'several iterations of editing.' (your answer)

**C** Keep all original source files unchanged in a reference directory so developers can manually trace any claim back to its origin

Preserving source files without explicit mappings forces developers to manually search through potentially hundreds of files to verify a single claim. This is not scalable and does not indicate which specific source informed which specific documentation statement.

- B** Maintain structured provenance metadata that maps each claim to its source file, line number, and retrieval date.

Structured provenance metadata survives editing because it is stored as data fields (e.g., JSON or YAML frontmatter) separate from the prose. When a developer questions a claim, they can trace it to the exact source file and line number. The retrieval date indicates whether the source was current when the documentation was generated. **(correct answer)**

- D** Version-control the documentation and use git blame to trace each line back to the commit that created it

git blame traces authorship of documentation lines to commits, not to the source material that informed those lines. A commit message might say 'update architecture guide' without indicating that a specific statement came from ADR-047 or a comment in auth-service/src/middleware.ts.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Q17 · D4 · 4.4 VALIDATION-RETRY / PYDANTIC-VALIDATION · q-4-4-010 · Structured Data Extraction Pipeline **CORRECT**

The pipeline validates each tool\_use extraction against a Pydantic model with model\_validate(). Validation succeeds and every field has the correct type, yet extracted totals still disagree with the documents' stated totals. What explains this, and what is the fix?

- D** Semantic disagreement indicates hallucination; lower the temperature so extraction becomes deterministic

Sampling settings do not create correctness, and determinism would only repeat the same wrong extraction; this failure class belongs to semantic validation.

- C** The schema needs numeric minimum and maximum constraints on the stated\_total field so that mismatched totals are rejected at parse time

A per-field range cannot express a relationship between two fields; the mismatch is only detectable by comparing them in validation code.

- A** model\_validate() is failing silently; pin an older Pydantic version until the regression is fixed

model\_validate() did its job: the extraction conforms to the model's types. Nothing failed silently; the missing check was never defined.

- B** Type checks passed but no rule relates the totals; add a model validator for the sum rule and route its errors into retries

Schema conformance and semantic correctness are different layers; the sum relationship must be written as a validator, whose errors drive the retry loop.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Pydantic as the validation layer)

Q18 · D5 5.6 INFORMATION-PROVENANCE / CLAIM-SOURCE-MAPPING q-5-6-002 · Multi-Agent Research System CORRECT

A multi-agent research system has three subagents (financial filings, news, technical white papers) and a synthesis agent. Each subagent returns properly attributed findings, but the final synthesised report has no source attribution: stakeholders cannot trace which claim came from which source. Which fix addresses the root cause?

D Store all subagent outputs in a database and have the synthesis agent reference database entries by ID instead of incorporating content directly

This adds infrastructure complexity without solving the synthesis problem. The synthesis agent still needs to merge and present findings coherently. Database references do not prevent attribution loss during the summarisation and rewriting that synthesis inherently requires.

A Append a bibliography section at the end of the report listing all sources each subagent consulted

A bibliography lists sources but does not map specific claims to specific sources. Stakeholders need to know which claim came from which document, not just which documents were consulted overall. This is document-level attribution, not claim-level provenance.

B Have each subagent include source URLs as inline hyperlinks in their prose output so the synthesis agent can preserve them

Inline hyperlinks in prose are fragile — the synthesis agent will rewrite, merge, and compress prose during summarisation, stripping or disconnecting links from their associated claims. Attribution must be in structured data, not embedded in prose that gets rewritten.

C Require subagents to output structured claim-source mappings and instruct the synthesis agent to preserve and merge them.

Structured claim-source mappings survive synthesis because they are data structures, not prose that gets rewritten. The synthesis agent can merge mappings from multiple subagents while preserving the link between each claim and its source. This also enables content-appropriate rendering: financial data as tables, news as prose, technical findings as lists — each with attribution intact.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation)

Q19 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-005 · Multi-Agent Research System CORRECT

A multi-agent system has subagents for web search, academic database, and industry reports. The academic database subagent hits a 403 Forbidden on a specific journal, catches the error, and returns an empty result set marked as successful. The coordinator produces a final report with no academic sources. What is the critical failure in this design?

**A** The coordinator should validate that all subagents return non-empty results before producing the final report.

Requiring non-empty results from all subagents would cause the workflow to fail when a subagent legitimately finds no matching data. The issue is that the error was suppressed, not that the coordinator failed to validate result volume.

**C** The subagent should have retried the request with exponential backoff, as 403 errors are often transient.

403 Forbidden is a permission error, not a transient error. Retrying will not resolve it — the subagent lacks the required access credentials. The correct response is to propagate the error with its type so the coordinator knows the failure is not retryable.

**B** The subagent silently suppressed the access failure by returning empty results as success, preventing the coordinator from attempting recovery or noting the gap in coverage.

Silent suppression — returning empty results marked as success — is the worst error propagation anti-pattern. The coordinator believes the academic search succeeded and found nothing, so it produces a report without academic sources and without noting the coverage gap. The subagent should have returned structured error context with the failure type, what was attempted, and potential alternatives.

**D** The entire workflow should have been terminated when the academic database subagent encountered the 403 error.

Workflow termination on a single subagent failure wastes partial results from the web search and industry reports subagents that may have completed successfully. The coordinator should proceed with available results and annotate the coverage gap.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Q20 · D4 · 4.4 VALIDATION-RETRY / RETRY-BOUNDARY · q-4-4-001 · Structured Data Extraction Pipeline **CORRECT**

Your extraction pipeline validates that line item amounts sum to the stated total. For Document A, the sum is £450 but the stated total is £500. For Document B, the 'department' field is missing entirely from the source text. Which retry strategy is correct?

**B** Retry Document A with the discrepancy error; flag Document B for human review since the information is absent from the source

Document A has a fixable discrepancy (the model may have missed a line item). Document B has genuinely absent information — retries are ineffective.

**A** Retry both: send the validation error for Document A and instruct the model to find the missing department for Document B

Document B's department is absent from the source. Retrying will not make information appear that does not exist.

**C**    Retry both documents with the same prompt since extraction is non-deterministic

Non-determinism does not create information that is absent. Document A may benefit from retry; Document B will not.

**D**    Skip retries for both and flag them for human review

Document A's discrepancy is likely fixable with error feedback. Skipping the retry wastes the model's self-correction capability.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry effectiveness boundary)

Q21 · D4    4.4 VALIDATION-RETRY / RETRY-WITH-ERROR    q-4-4-002 · Structured Data Extraction Pipeline    **CORRECT**

Your extraction pipeline retries failed documents, but the retry simply resends the original document with the same prompt. Success rates on retries are only marginally better than the first attempt. Developers have identified that most failures are format mismatches where the model places values in the wrong fields. How should you improve the retry mechanism?

**D**    Send the original document, the failed extraction, and the validation error naming the misplaced fields.

Retry-with-error-feedback is dramatically more effective than naive retries. Sending the original document, the failed extraction, and the specific validation error allows the model to self-correct. Format mismatches are fixable errors that respond well to this approach.

**B**    Switch to a different model for retries so a fresh perspective catches the errors

The issue is not the model but the absence of error context. Any model retrying without seeing the specific validation error will likely repeat the same mistakes.

**C**    Skip retries and route all failures directly to human review to avoid wasting API costs

Format mismatches are fixable errors. The model can self-correct when given proper error feedback. Skipping retries wastes the model's self-correction capability for errors that are resolvable.

**A**    Increase the number of retries from 1 to 3, since extraction is non-deterministic and more attempts improve odds

Naive retries without error feedback produce the same mistakes. More attempts at the same flawed approach yield diminishing returns. The model needs to see what went wrong.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q22 · D1    1.6 TASK-DECOMPOSITION / ADAPTIVE-DECOMPOSITION    q-1-6-009 ·



Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

You are planning how Claude should tackle a large, unfamiliar refactoring effort. Which characteristics of the work indicate dynamic adaptive decomposition rather than a fixed sequential pipeline? (Select 2)

D

The task is open-ended, like adding comprehensive tests to a legacy codebase you have not mapped yet.

This is the guide's example of open-ended work: map the structure first, then build a prioritised plan that adapts as dependencies surface.

B

The useful subtasks only become clear as intermediate findings come in.

Adaptive plans generate subtasks from what each step discovers, which a fixed pipeline cannot do.

A

The review covers the same predictable set of aspects on every run.

A predictable multi-aspect review is exactly where a fixed prompt-chaining pipeline fits best.

C

Each step's output feeds the next in a stable, known order.

A stable, known step order is the defining property of a sequential pipeline, not of adaptive decomposition.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition)

Lesson 1.6: Task Decomposition Strategies (Sequential pipelines)

Q23 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CONTEXT q-2-2-011 · Multi-Agent Research System

CORRECT

A research subagent's database query fails partway through an investigation. Which pieces of information belong in the structured error context it returns to the coordinator? (Select 3)

C

The failure type, such as transient, validation, or permission.

Categorising the failure lets the coordinator make the right retry-or-reroute decision.

E

A plain 'no results found' message in place of the error.

Reporting an access failure as an empty result conflates two different situations and hides the need for a retry decision.

D

What was attempted, including the query that failed.

The attempted query gives the coordinator enough detail to retry intelligently or reformulate the request.

B

Partial results gathered before the failure.

Partial results preserve completed work so the coordinator does not re-run what already succeeded.

**A** No report at all: the subagent should retry silently until the query succeeds.

Silent retries hide failures from the coordinator, which owns error handling and needs observability into what went wrong.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

MCP: Tools Specification ↗

Q24 · D2 · 2.3 TOOL-DISTRIBUTION-CHOICE / CONSOLIDATION · q-2-3-008 INCORRECT

The data platform team has built an MCP server with 22 tools: `query_snowflake`, `query_postgres`, `query_api`, plus 19 specialised tools for individual data transformations (`pivot_table`, `calculate_percentile`, `normalise_currency`, etc.). Agents take 3-4 turns to select the *correct* tool and frequently choose the *wrong* transformation. What is the most effective redesign?

**A** Improve all 22 tool descriptions with detailed examples and boundary conditions to help the agent distinguish between them.

Better descriptions help, but 22 tools still exceeds the practical limit for reliable selection. Research shows tool selection degrades significantly beyond 4-5 tools per agent. The tool count itself is the problem.

**C** Use `tool_choice: 'any'` to force the agent to always call a tool, eliminating turns where the agent reasons without acting.

Forcing tool calls does not improve selection accuracy — it just ensures the agent picks something, potentially the wrong tool. The root cause is too many similar tools, not too few tool calls.

**D** Split the tools across two separate MCP servers — one for queries and one for transformations — to reduce cognitive load.

MCP server boundaries are invisible to the agent. All tools from all connected servers appear in a single list. Splitting across servers does not reduce the number of tools the agent must choose from. (your answer)

**B** Consolidate the 19 transformation tools into a single `transform_data` tool with a `transform_type` parameter, reducing the total to 4 tools.

Reducing from 22 to 4 tools brings the count within the optimal 4-5 range for reliable tool selection. The 19 transformation tools share a common pattern (input data, transformation type, output format) and are natural candidates for consolidation into a single parameterised tool. (correct answer)

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Consolidating near-duplicate tools)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q25 · D1 · 1.3 SUBAGENT-INVOCATION-CONTEXT / GOAL-ORIENTED-PROMPTS · q-1-3-018 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A coordinator delegates a large refactoring subtask to a specialist subagent, and the architect must write the subagent's prompt. Which prompt design best follows the guide's guidance on how a coordinator should instruct a subagent?

D

Omit the goal and instead list the tools the subagent may use, letting it infer the objective from the toolset

Listing tools without stating the goal leaves the objective ambiguous, and the subagent may optimise for the wrong outcome. The prompt must state the goal.

C

Provide only the high-level goal with no success criteria, so the subagent has maximum freedom

A goal with no quality criteria gives the subagent no way to know when it has actually succeeded. Effective goal-oriented prompts pair the objective with explicit success criteria.

B

Give an exact step-by-step procedure (open file X, change line 12, run command Y, then edit file Z) so the subagent cannot deviate

A rigid procedure constrains the subagent and breaks the moment reality diverges from the script. The guide specifically warns against procedural instructions in favour of goal-oriented ones.

A

State the goal and its quality criteria (no breaking API changes, all tests pass) and let the subagent choose how to get there.

Goal-oriented prompts specify what to achieve and the quality bar, not a fixed procedure. Pairing the objective with success criteria lets the subagent adapt its approach when it meets something unexpected, which is what the guide recommends.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Goal-oriented prompts)

Q26 · D3 · 3.6 CI/CD-INTEGRATION / WORKTREE-PARALLEL · q-3-6-008

CORRECT

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

The team wants three Claude Code instances working in parallel: one extracting the authentication service, one extracting the billing service, and one updating shared libraries. All three need to commit to the same repository without conflicts. What is the correct setup?

A

Run all three instances in the same working directory on separate branches, switching branches as needed

Multiple Claude Code instances in the same working directory will cause file system conflicts, dirty working trees, and race conditions, even on different branches. Git branch switching affects the entire working directory.

C

Clone the repository three times into separate directories and merge the results manually

Three separate clones create divergent git histories that must be reconciled manually. git worktree provides the same isolation with a shared repository, making merges straightforward through normal branch operations.

**B** Use git worktree to create three separate working directories, each on its own branch, with one Claude Code instance per worktree

git worktree creates multiple working directories from the same repository, each checked out to a different branch. Each Claude Code instance operates in its own isolated file system while sharing the same git history. This eliminates file system conflicts between parallel instances.

**D** Use fork\_session to run the three tasks as parallel branches within a single Claude Code session

fork\_session creates parallel exploration branches within Claude Code's conversation context, not separate file system environments. Three large extraction tasks each need their own working directory to avoid file conflicts, which requires git worktree.

WHERE THIS COMES FROM

[Anthropic: Claude Code Documentation](#) ›

[Git: Worktrees](#) ›

Q27 · D2 2.4 MCP-SERVER-INTEGRATION / BUILD-VS-USE q-2-4-007 CORRECT

An enterprise data platform team is evaluating how to integrate with their existing PostgreSQL database, Slack workspace, and a custom internal approval workflow. A developer proposes building three custom MCP servers. What is the correct approach?

**A** Build all three custom MCP servers to ensure tight integration with the team's specific requirements and coding standards.

Building custom servers for PostgreSQL and Slack is unnecessary when well-maintained community servers already exist for these standard integrations. Custom builds should be reserved for genuinely unique workflows.

**B** Use community MCP servers for PostgreSQL and Slack, and build a custom server only for the internal approval workflow that has no community equivalent.

Community servers should be evaluated first for standard integrations. PostgreSQL and Slack have well-maintained community MCP servers. The internal approval workflow is team-specific with no community equivalent, making it the only justified custom build.

**D** Use community MCP servers for all three integrations, adapting the internal approval workflow to fit an existing community server's interface.

Forcing a custom internal workflow into a community server's interface leads to awkward abstractions and missing functionality. The community-first principle applies to standard integrations, not to genuinely unique team-specific workflows.

**C** Skip MCP entirely and use direct API calls from Bash for all three integrations to avoid the overhead of running MCP servers.

Direct API calls bypass the MCP tool interface, losing the benefits of structured tool descriptions, standardised error handling, and agent-native integration. MCP servers provide a consistent interface that agents understand natively.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Build-vs-use)

Claude Code: MCP Server Configuration ↗

Q28 · D2 · 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY · q-2-2-001 CORRECT

A tool returns an empty array after a customer lookup. The agent retries 3 times, then escalates to a human agent. Analysis shows the customer's account simply does not exist. What is the root cause of this wasted effort?

A The retry limit is too low — increasing to 5 retries would resolve the issue.

More retries make the problem worse. The tool succeeded — it found no matching customer. Retrying a successful query with no matches will never produce different results.

B The tool does not distinguish between access failures and valid empty results, so the agent treats 'no matches' as a retrievable failure.

The tool successfully queried the data source and found no matches. This is a valid empty result, not an access failure. The agent's recovery logic cannot distinguish the two, causing it to retry when it should accept the result.

C The escalation threshold is too aggressive — the agent should exhaust more retries before involving a human.

The problem is not the escalation threshold. The problem is that the agent retries at all. A valid empty result requires no retry and no escalation.

D The system prompt should instruct the agent not to retry customer lookups.

Hard-coding retry rules per tool in the system prompt is brittle. The proper fix is structured error metadata that tells the agent whether the result is retryable.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q29 · D1 · 1.6 TASK-DECOMPOSITION / COORDINATOR · q-1-6-007 · Multi-Agent Research System CORRECT

A multi-agent research system must process a customer's request that involves three sequential stages: data collection, analysis, and report generation. Each stage depends on the output of the previous one. Which orchestration pattern is most appropriate?

B Pipeline orchestration — pass the output of each stage as input to the next in a defined sequence

Pipeline orchestration is the correct pattern for sequential dependencies. Each stage completes before the next begins, with the output of one stage serving as the input to the next. This matches the data collection → analysis → report generation dependency chain.

**A** Parallel orchestration — run all three subagents simultaneously to minimise latency

Parallel execution requires independent tasks. Here, analysis depends on data collection results and report generation depends on analysis results. Running them simultaneously would mean the analysis agent has no data to analyse.

**D** Hub-and-spoke with all three agents reporting independently to the coordinator

Independent reporting ignores the sequential dependencies. The analysis agent cannot produce meaningful results without data collection output. The pipeline must enforce the execution order.

**C** Dynamic adaptive decomposition — let the coordinator decide the order at runtime based on query complexity

Dynamic adaptive decomposition is for open-ended investigation tasks where the next step depends on discoveries. Here, the three stages and their dependencies are known in advance, making a fixed pipeline the more predictable and efficient choice.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q30 · D1 1.2 ORCHESTRATION-PATTERNS / DECOMPOSITION q-1-2-006 · Multi-Agent Research System CORRECT

A research system receives a customer query: 'Compare the environmental impact and cost-effectiveness of electric vehicles versus hydrogen fuel cell vehicles for commercial fleet operations.' The fleet manager needs a like-for-like comparison, with each dimension measured consistently across both vehicle types. The coordinator must decide how to decompose this task. Which decomposition strategy is most appropriate?

**A** Single-agent approach: assign the entire comparison to one research agent with access to all tools

A single agent with all tools violates scoped tool access and would struggle with attention dilution across the multiple dimensions of this comparison (environmental impact, cost-effectiveness, two vehicle types, commercial fleet context).

**B** Parallel tracks by dimension (environmental impact, cost-effectiveness), each covering both vehicle types, then synthesise.

The query has two distinct analytical dimensions (environmental impact and cost-effectiveness) applied to the same subjects. Multi-concern decomposition splits these into parallel tracks, each with its own subagent. Both tracks investigate both vehicle types, ensuring consistent comparison methodology. A synthesis step produces the unified comparison.

**C** Vehicle-based decomposition: assign one agent to research EVs and another to research hydrogen fuel cells, then merge findings

Splitting by vehicle type risks inconsistent comparison methodology. One agent might emphasise different environmental metrics than the other, making the comparison unreliable. Splitting by analytical dimension (environmental impact, cost-effectiveness) ensures each dimension uses consistent methodology across both vehicle types.

**D** Dynamic adaptive decomposition: start with a broad search and let the coordinator decide next steps based on what it finds

The query's structure is clear and predictable — two analytical dimensions (environmental, cost), two vehicle types, one context (commercial fleets). Dynamic adaptive decomposition is for open-ended investigations where the next steps are unknown. Here, multi-concern decomposition is more efficient.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q31 · D4 · 4.4 VALIDATION-RETRY / INTER-STEP-VALIDATION · q-4-4-006 · Structured Data Extraction Pipeline **CORRECT**

Your document processing pipeline chains three steps: (1) extract raw text, (2) classify document type, (3) extract structured fields based on type. Step 2 occasionally misclassifies invoices as purchase orders, causing step 3 to extract the wrong fields. The team proposes combining steps 2 and 3 into a single prompt to reduce errors. What is the better approach?

**C** Add more few-shot classification examples to step 2's prompt so misclassifications stop occurring

Better examples may help, but without validation between steps, misclassifications still cascade silently into step 3. Inter-step validation is the structural fix.

**D** Run step 2 three times and use majority voting to determine the document type

Majority voting adds cost and latency without addressing the root cause. Validation between steps is more efficient and catches the specific failure mode.

**B** Keep steps separate and validate the classification between steps 2 and 3 before extraction

Keeping steps separate maintains focused prompts. Adding validation between steps catches misclassifications before they cascade into wrong-field extraction. This is the core principle of prompt chaining: focused steps with inter-step validation.

**A** Combine steps 2 and 3 as proposed, since fewer steps means fewer failure points

Combining steps creates a less focused prompt that must handle both classification and extraction simultaneously. This increases attention dilution and makes it harder to diagnose which part fails.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Q32 · D5   5.3 ERROR-PROPAGATION / COVERAGE-ANNOTATIONS   q-5-3-007 · Technical Documentation Maintenance System

CORRECT

A documentation generation pipeline uses Claude Code to produce docs from three sources: source code comments, existing wiki pages, and API schema files. During a run, the wiki page retrieval fails with a timeout, but the source code and API schema are available. The pipeline currently halts entirely on any source failure. What is the best error handling strategy?

A   Retry the wiki retrieval three times with exponential backoff. If all retries fail, halt the pipeline to prevent incomplete documentation

Halting the entire pipeline because one of three sources is unavailable wastes the successful results from the other two sources. Documentation from source code and API schemas is still valuable even without wiki content.

D   Use the source code comments to infer what the wiki pages would have contained, filling in the gaps with generated content

Inferring wiki content from source code produces hallucinated documentation that appears authoritative. Wiki pages often contain operational context, known issues, and tribal knowledge that cannot be inferred from code.

C   Silently skip the wiki source and generate documentation from the remaining two sources without noting the omission

Silent omission hides the failure. Users would trust the documentation as complete, not knowing that wiki-sourced context (which may contain critical operational notes or caveats) is missing.

B   Return structured error context for the wiki failure, proceed with available sources, and mark the gaps that lack wiki content.

This produces maximum value from available sources while maintaining transparency about what is missing. Structured error context enables intelligent recovery (retry later, alert the team). Explicit gap markers prevent users from trusting incomplete documentation as complete.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Coverage annotations)

Q33 · D3   3.1 CLAUDE-MD-HIERARCHY / PRECOMPACT-HOOK   q-3-1-018 · Technical Documentation Maintenance System

INCORRECT

A documentation team wants to archive the full conversation transcript to a log file every time Claude Code runs /compact, so that context lost during compaction can be reviewed later. Which hook configuration achieves this?

B   A PreCompact hook that writes the current transcript to a timestamped log file before /compact summarises the conversation

PreCompact is a first-class hook event in Claude Code that fires immediately before /compact (or auto-compaction) runs. The hook receives the full pre-compaction transcript path and can copy or process it before summarisation



discards detail. This is the documented mechanism for exactly this use case. (correct answer)

**A** A PostToolUse hook on Write that snapshots the transcript whenever a file is written, on the assumption /compact will eventually run

Write and compaction are unrelated. PostToolUse on Write would fire on every file write whether or not /compact is about to run, producing noise rather than a transcript at the right moment. The dedicated PreCompact event is the correct hook.

**C** A PreToolUse hook configured on a built-in 'Compact' tool, matching tool name 'Compact'

There is no 'Compact' tool exposed to PreToolUse. Compaction is a Claude Code lifecycle operation, not a model-invoked tool, so it has its own dedicated PreCompact event rather than being hooked via PreToolUse. (your answer)

**D** A PostToolUse hook on all tools that appends each tool result to a running log file, creating a continuous archive

Logging every tool result creates enormous log files and does not capture conversation context (reasoning, planning, user messages). The targeted PreCompact event captures the full transcript at the critical moment without per-tool noise.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q34 · D4 · 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-COVERAGE · q-4-2-001 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline correctly identifies data in structured tables but returns empty fields when the same information appears in narrative paragraphs. Detailed instructions specify all required fields. What should you try first?

**C** Add a pre-processing step to convert all narrative text into tables

This adds unnecessary complexity. The model should handle varied formats — few-shot examples teach it how.

**B** Add few-shot examples showing correct extraction from both structured tables and narrative paragraphs

Few-shot examples demonstrating correct handling of varied document structures directly address inconsistent extraction quality across formats.

**D** Add a post-processing retry that re-extracts any empty fields

Retrying without better guidance will produce the same empty results. The model needs examples of correct extraction from narrative text.

**A** Increase the model's context window to process more of the document

The model already finds data in tables. The issue is not context size but inconsistent handling of different document structures.

WHERE THIS COMES FROM

- Lesson 4.2: Few-Shot Prompting (Few-shot covering varied input)
- Lesson 4.2: Structured Output with Tool Use (What tool\_use does not enforce)

Q35 · D4 · 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-AMBIGUITY · q-4-2-007 · Structured Data Extraction Pipeline CORRECT

An invoice-extraction pipeline mislabels fields on documents where the correct interpretation is ambiguous. Which few-shot practices should you apply? (Select 2)

B

Use examples showing correct extraction from varied document structures to fix empty or null extraction of required fields.

Varied-structure examples fix empty or null extraction of fields that are present, which is the few-shot remedy for unreliable extraction; fabrication of absent data is handled by nullable schema fields instead.

D

Create two to four targeted examples for the ambiguous cases, showing why one interpretation is chosen.

Targeted examples that demonstrate the reasoning are the guide's specific remedy for ambiguous cases.

A

Add an example for every document variant seen in production so far.

Volume is not the fix; a handful of targeted examples for the ambiguity beats exhaustive coverage and its context cost.

C

Replace the examples with a longer natural-language description of each field.

Descriptions alone are what produced the inconsistency; examples communicate the expected mapping more reliably than more prose.

WHERE THIS COMES FROM

- Lesson 4.2: Few-Shot Prompting (Few-shot construction)
- Anthropic: Multishot (Few-Shot) Prompting ↗

Q36 · D3 · 3.1 CLAUDE-MD-HIERARCHY / ENFORCEMENT · q-3-1-006 · Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

The refactoring team's Java naming convention must be applied on every run. A senior developer's user-level CLAUDE.md (~/.claude/CLAUDE.md) contradicts it, and reviewers have already caught Claude following the developer's personal preference over the team rule. Where should the team move the rule so it is guaranteed to be honoured?

A

Into the project's settings.json or a PreToolUse hook — enforcement-grade locations that the Claude Code client applies regardless of what the model decides

Anthropic's memory docs are explicit that CLAUDE.md files are 'concatenated into context rather than overriding each other' and that 'if two rules contradict each other, Claude may pick one arbitrarily.' CLAUDE.md is delivered as a user message and the docs warn there's 'no guarantee of strict compliance.' The docs themselves point teams at the right alternative: 'Settings rules are enforced by the client regardless of what Claude decides to do. CLAUDE.md instructions

shape Claude's behavior but are not a hard enforcement layer.' For a rule that must hold every time, move it to settings.json permissions or a hook — hooks run as shell commands at fixed lifecycle events and apply 'regardless of what Claude decides to do.'

**C** Promote the rule into the user-level CLAUDE.md of every team member so it loads first and sets the baseline

User-level CLAUDE.md is not version-controlled or shared via git, and the docs treat it as the broadest scope, not the highest priority. Asking every teammate to maintain a personal copy of a team rule is fragile and still gives no enforcement guarantee — contradictory rules can still resolve arbitrarily.

**B** Leave the rule in the repository-root CLAUDE.md — the more specific scope wins on conflicts, so it will override the user-level file

This is the popular paraphrase, but the official docs never assert it. They describe a load order (broadest scope to most specific, so project instructions appear in context after user instructions) but explicitly say files are 'concatenated into context rather than overriding each other' and conflicts 'may [be] pick[ed] arbitrarily.' Treating CLAUDE.md scope as a precedence chain is exactly the assumption that has already failed the team in the scenario.

**D** Keep the rule in CLAUDE.md but move it into a CLAUDE.local.md so it appends last and overrides the personal preference

Load order does put CLAUDE.local.md after CLAUDE.md within a directory, but the docs are careful to call this 'load order,' not precedence — they still say 'if two rules contradict each other, Claude may pick one arbitrarily.' CLAUDE.local.md is also gitignored, so a team rule belongs there even less than in the shared CLAUDE.md.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Loading order and conflict handling)

Anthropic — How Claude remembers your project (memory docs) ↗

Q37 · D5 5.5 HUMAN-REVIEW-CALIBRATION / REVIEWER-CAPACITY q-5-5-007 ·

CORRECT

Technical Documentation Maintenance System

Claude Code generates API reference pages for 150 endpoints across three categories: payment processing (30 endpoints, high regulatory risk), internal tooling (80 endpoints, low risk), and public data queries (40 endpoints, moderate risk). The team cannot manually review all 150. How should they structure human review?

**A** Randomly sample 15% of all endpoints (approximately 23 pages) for review, ensuring a representative cross-section

Uniform random sampling treats all endpoints equally, but payment processing endpoints carry much higher risk. A 15% sample might only include 4-5 payment endpoints, inadequately covering the highest-risk category.

**C** Review only the payment processing endpoints since they are the highest risk, and trust Claude Code's output for the remaining 120 endpoints

Completely skipping review for 120 endpoints risks undetected errors in public-facing documentation. Even low-risk internal tooling docs should have some sample review to catch systematic generation issues that might affect all categories.

**B** Use stratified sampling: review 100% of payment docs, 10% of public data query docs, and 5% of internal tooling docs.

Stratified sampling allocates review effort proportional to risk. Payment processing documentation has regulatory implications warranting full review. Public data queries affect external users and deserve moderate scrutiny. Internal tooling has the lowest blast radius and can tolerate the lightest review. This achieves thorough coverage of critical content within a manageable review budget.

**D** Have Claude Code self-assess each generated page with a confidence score and only review pages where confidence falls below 80%

LLM self-reported confidence is poorly calibrated. Claude Code may report high confidence on pages with subtle factual errors (e.g., incorrect parameter types, wrong default values) because the generated text is fluent and internally consistent.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Reviewer capacity)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q38 · D5 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING q-5-5-002 · Structured Data Extraction Pipeline

CORRECT

A legal document extraction pipeline has been running for three months with calibrated confidence thresholds. Extractions above 90% confidence are auto-approved without human review, saving 60% of reviewer time. A new client submits contracts written in an unusual two-column legal format the system has not encountered before. The system reports 92% confidence on these extractions. What is the appropriate safeguard?

**B** Trust the calibrated confidence threshold since it was validated against labelled data and the system reports 92% confidence, which exceeds the 90% auto-approval threshold

Confidence calibration is only valid for document types represented in the calibration data. A novel format the system has never encountered can produce miscalibrated confidence scores — the model may be systematically overconfident because it does not recognise its own unfamiliarity with the format.

**C** Raise the confidence threshold from 90% to 98% for all document types to account for novel formats

Raising the threshold globally penalises well-understood document types that are genuinely accurate at 90%+. This wastes reviewer capacity on documents that do not need review, while still not guaranteeing detection of format-specific issues.

**D** Add a document format classifier that flags unknown formats for mandatory human review before any auto-approval

While not wrong in principle, this adds ML infrastructure complexity that stratified sampling already handles. A format classifier requires training data and maintenance, whereas ongoing stratified sampling detects novel error patterns without additional models.

**A** Maintain stratified random sampling of high-confidence extractions across document types to catch the new format's error rate.

Stratified random sampling is designed precisely for this scenario: detecting novel error patterns in high-confidence extractions that slip past threshold-based routing. By sampling across document types, the new format's errors would surface during routine verification even though the confidence scores appear acceptable.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q39 · D2 2.5 BUILT-IN-TOOLS / GREP-THEN-EDIT q-2-5-006 **INCORRECT**

An engineering team needs to update all references to a renamed API endpoint across their 500,000-line codebase. They know the old endpoint name (POST /api/v1/users/create) but not which files reference it. After locating the files, they need to replace the old endpoint with the new one (POST /api/v2/users). What is the correct tool sequence?

**D** Use Bash with sed to perform a global find-and-replace across all files in a single command.

A global sed command bypasses the built-in tools designed for this task and risks unintended modifications without the ability to review each change. Edit provides targeted modifications with unique text matching for safety.

**B** Use Read on the API router file to find all endpoint definitions, then manually trace each reference.

This approach only finds where the endpoint is defined, not where it is referenced. It also requires knowing which file contains the router definition, and manual tracing is unreliable across a large codebase.

**A** Use Glob for '\*\*/\*.md' to find all documentation files, then Read each one to check for the old endpoint, then use Edit to update matches.

Glob finds files by path pattern, but the endpoint reference could appear in any file type, not just Markdown. Reading every documentation file to search for the string wastes context tokens on irrelevant files. (your answer)

**C** Grep for '/api/v1/users/create' to find every file with the old endpoint, then use Edit to replace it in each match.

Grep searches file contents across the entire codebase, locating all references regardless of file type or location. Edit then makes targeted replacements in each matched file. This is the minimal, precise approach. (correct answer)

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep for content)

Lesson 2.5: Built-in Tools (Edit replacements)

Q40 · D3 3.6 CICD-INTEGRATION / WORKTREE-PARALLEL q-3-6-010 · Technical Documentation Maintenance System

**CORRECT**

A documentation team needs to simultaneously update API reference docs for three independent microservices after a breaking change. Each update requires reading source code, updating Markdown files, and validating links. A single Claude Code session would exhaust the context window trying to hold all three services' code simultaneously. What is the recommended approach?

**A** Process the three services sequentially in the same session, running /compact between each service to free context

Even with /compact, each service's context (source code, documentation, edits) is substantial. Sequential processing risks context degradation as compaction summaries lose detail from earlier services, and it is slower than parallel processing.

**B** Use git worktree for three branches, each with its own Claude Code session on one service, then merge the results

git worktree creates separate working directories on different branches, each with its own isolated Claude Code session. Each session has a full context budget dedicated to one service. The three updates run in parallel without interfering with each other, and results are merged via standard git workflows.

**D** Split the documentation files into smaller chunks and process each chunk in a separate API call using the batch API

The batch API is for high-throughput, latency-tolerant workloads — not for interactive Claude Code sessions. Documentation updates require interactive exploration of source code and iterative editing, which the batch API does not support.

**C** Create a single skill with context: fork that processes all three services in parallel within one session

context: fork isolates a skill's output from the main conversation but does not create multiple parallel execution contexts. A single skill cannot simultaneously process three independent services in parallel.

WHERE THIS COMES FROM

Anthropic: Claude Code Documentation ↗

Git: Worktrees ↗

Q41 · D4 · 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING · q-4-3-006 · Structured Data Extraction Pipeline CORRECT

Your data extraction pipeline uses tool\_use with tool\_choice set to 'auto'. You need guaranteed structured output for every request, but some documents occasionally produce conversational text responses instead. You have only one extraction tool defined. What is the most reliable fix?

**D** Switch to prompt-based JSON extraction with strict formatting instructions for more predictable output

Prompt-based JSON is less reliable than tool\_use. It risks syntax errors and missing fields. Forcing the tool via tool\_choice is the correct approach.

**C** Add a post-processing step that retries any request returning text instead of a tool call

Retrying with the same 'auto' tool\_choice may produce the same text response. Fixing tool\_choice eliminates the problem at the API level rather than patching it after the fact.

**A** Add a system prompt instruction: 'You must always use the extraction tool and never produce text responses'

System prompt instructions cannot guarantee tool use when tool\_choice is 'auto'. The API-level tool\_choice parameter is the authoritative mechanism.

**B** Set tool\_choice to force the specific extraction tool by name

Setting tool\_choice to {type: 'tool', name: 'extract\_data'} forces the model to call that specific tool on every request, guaranteeing schema-compliant structured output with no possibility of text-only responses.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (tool\_choice modes)

Anthropic: Tool Use Documentation ↗

Q42 · D1 · 1.6 TASK-DECOMPOSITION / ADAPTIVE-DECOMPOSITION · q-1-6-002 · Multi-Agent Research System · CORRECT

A consulting firm needs an agent to add a comprehensive test suite to a large legacy codebase. The codebase has no existing tests, unclear dependencies between modules, and the team does not know which areas are most critical. Which task decomposition strategy is most appropriate?

**C** Dynamic adaptive decomposition: map the codebase, find the high-impact areas, and adapt the plan as dependencies emerge.

Adding tests to a legacy codebase with unknown dependencies is an open-ended investigation task. Dynamic adaptive decomposition generates subtasks based on what is discovered at each step — first mapping the structure, then identifying critical areas, then adapting the test plan as dependency relationships emerge.

**D** A multi-pass architecture with per-file analysis and a cross-file integration pass, exactly like a standard code review pipeline.

Multi-pass architecture solves attention dilution for review tasks, but test generation in a legacy codebase needs adaptive exploration: discovering critical modules and dependencies and prioritising coverage. The decomposition must adapt to findings, not follow a fixed per-file pattern.

**A** A fixed sequential pipeline that reviews each file in alphabetical order and generates tests for each one

Fixed sequential pipelines are best for predictable, structured tasks. Adding tests to a legacy codebase with unclear dependencies is an open-ended investigation that requires adaptive exploration to discover which areas are most critical and how dependencies affect test design.

- B** A single-pass analysis that processes the entire codebase at once and generates a complete test plan

A single-pass analysis of a large codebase would suffer from attention dilution, producing inconsistent coverage. Additionally, a single pass cannot adapt to discoveries about dependencies and critical areas that emerge during exploration.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition)  
Lesson 1.6: Task Decomposition Strategies (Pattern selection)

Q43 · D3 · 3.6 CI/CD-INTEGRATION / STRUCTURED-VALIDATION · q-3-6-007 · Technical Documentation Maintenance System

CORRECT

A CI pipeline validates that generated API docs match the codebase. It runs ``claude -p 'Verify that all public API endpoints in src/api/ have corresponding documentation in docs/api/'`` with ``--output-format json``. The check consistently reports 100% coverage, yet manual audits regularly find undocumented endpoints. What is the most likely cause?

- C** The `-p` flag prevents Claude Code from reading files, so it generates a plausible-sounding report without actual file access

The `-p` flag runs Claude Code in non-interactive (pipe) mode. It does not restrict file access — Claude Code retains full access to its built-in tools (Read, Grep, Glob) for reading files.

- D** The CI pipeline needs a separate review step where a second Claude Code instance verifies the first instance's findings

Adding a second instance to review the first adds latency and cost without addressing the root cause. If the first instance's prompt lacks clear validation criteria, the second instance reviewing the same vague output will draw the same flawed conclusions.

- A** The `--output-format json` flag corrupts the analysis results, causing false positives

`--output-format json` controls the output structure, not the analysis logic. It formats results as JSON for downstream parsing but does not affect the accuracy of the verification itself.

- B** Claude Code is doing a shallow pattern match, not a deep semantic comparison, and the prompt lacks validation criteria

Without explicit validation criteria (e.g., 'for each exported function in `src/api/`, check that `docs/api/` contains a section with the function name, parameters, and return type'), the model may perform a shallow check. Adding structured criteria, expected output format, and examples of what constitutes a gap produces reliable verification.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output and validation)  
Lesson 3.6: CI/CD Integration (CLAUDE.md for CI context)



Q44 · D2 · 2.2 STRUCTURED-ERROR-RESPONSES / TRANSIENT-ERROR · q-2-2-005 · Multi-Agent Research System CORRECT

A research agent calls an external API via an MCP server. After the 30th query in a batch of 50, the API starts returning HTTP 429 errors. The MCP server returns a generic 'Request failed' for every failure, so the agent abandons the batch after three consecutive failures. What MCP server change would most improve resilience?

**B** Return `errorCategory: 'transient'`, `isRetryable: true`, with a `retryAfterMs` field telling the agent how long to wait.

Rate limiting is a transient error — it will resolve after a delay. Structured metadata with the specific retry delay lets the agent space out remaining queries intelligently rather than abandoning the batch. The agent can continue processing other tasks while waiting.

**A** Implement automatic retry with exponential backoff inside the MCP server, hiding rate limits from the agent entirely.

Hiding rate limits from the agent removes its ability to make informed decisions, such as prioritising remaining queries, switching to cached results, or reporting partial progress. The agent should receive structured metadata to decide its own recovery strategy.

**C** Return `errorCategory: 'business'`, `isRetryable: false` to tell the agent to stop making requests entirely.

Rate limits are transient (they resolve after a delay), not business errors (which represent policy or rule violations the agent cannot recover from). Marking them as non-retryable causes the agent to abandon 20 remaining queries that would succeed after a brief delay.

**D** Queue all 50 requests at the MCP server level and process them sequentially with built-in rate limiting.

Server-side queuing removes the agent's ability to prioritise, cancel, or reorder queries based on intermediate results. It also creates a long-running blocking call that prevents the agent from doing other work.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Transient errors and retry-after)

Q45 · D2 · 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CATEGORIES · q-2-2-006 · Multi-Agent Research System CORRECT

A 'search\_papers' MCP tool has three failure patterns: (1) the upstream academic API returns HTTP 503 during peak hours, (2) users request papers from a restricted journal the system has no licence for, and (3) the agent submits a malformed DOI like 'doi-1234' that fails the input regex. The team wants structured error metadata so agents can handle each case differently. Which 'errorCategory' and 'isRetryable' combination is correct for all three?

**B** HTTP 503: transient/retryable; restricted journal: business/not retryable; malformed DOI: validation/retryable.

HTTP 503 is a temporary upstream outage (transient, will likely resolve on retry). Restricted journal is a policy limitation (business, will never resolve on retry — escalate or offer an alternative source). Malformed DOI is validation; the agent

must repair the input and then retry.

**D** HTTP 503: validation/retryable; restricted journal: business/not retryable; malformed DOI: business/not retryable.

HTTP 503 is transient (a temporary upstream outage), not validation — nothing is wrong with the request input. A malformed DOI is validation (fix input and retry), not business — business errors represent policy or rule violations, not input format problems.

**A** All three should be `errorCategory: 'transient'`, `isRetryable: true`, since they all prevent the tool from completing its task.

Lumping every failure into 'transient' is the canonical anti-pattern. The restricted journal case is a business error (licence policy); the agent must escalate, not retry. The malformed DOI is validation; the agent must repair the input format first.

**C** HTTP 503: transient/retryable; restricted journal: transient/retryable; malformed DOI: transient/retryable.

The restricted journal error is not transient — the system lacks a licence, and no amount of retrying will grant access. A malformed DOI is validation, not transient; the agent must repair the input rather than blindly retry the same value.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Q46 · D5 · 5.3 ERROR-PROPAGATION / STRUCTURED-ERROR-CONTEXT · q-5-3-001 · Multi-Agent Research System · CORRECT

A web search subagent in a multi-agent research system times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which approach best enables intelligent recovery?

**B** Implement automatic retry with exponential backoff, returning a generic 'search unavailable' status only after all retries are exhausted

The generic status hides valuable context from the coordinator, preventing informed recovery decisions.

**D** Propagate the timeout exception to a top-level handler that terminates the entire research workflow

Workflow termination wastes partial results from other subagents that may have completed successfully.

**A** Return structured error context including failure type, attempted query, partial results, and potential alternative approaches

This gives the coordinator everything it needs to decide: retry with modified query, try an alternative approach, or proceed with partial results.

**C** Catch the timeout and return an empty result set marked as successful

Silent suppression prevents any recovery. The coordinator believes the search succeeded and found nothing, so it will not attempt alternatives.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q47 · D1 · 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS · q-1-3-017 · Multi-Agent Research System · CORRECT

A web search agent has 9 tools: 'web\_search', 'url\_fetch', 'html\_parse', 'pdf\_extract', 'image\_ocr', 'translate', 'summarise', 'keyword\_extract', 'sentiment\_analysis'. In testing it frequently calls 'summarise' and 'sentiment\_analysis' when it should only fetch raw data. How should the architect fix this?

C Keep all 9 tools but implement PreToolUse hooks that block the web search agent from calling summarise and sentiment\_analysis

Using hooks to block tool access is an over-engineered workaround for what should be a configuration fix. The proper solution is to remove the tools from allowedTools, not to provide them and then block their use.

B Reduce the web search agent to the 4-5 data-fetching tools and move the analysis tools to the specialist agents.

The recommended maximum is 4-5 tools per agent. The web search agent has 9 tools, causing it to use tools outside its intended role. Reducing to data-fetching tools enforces scoped access. The analysis tools (summarise, sentiment\_analysis, keyword\_extract) belong with the synthesis or analysis agents.

A Add system prompt instructions telling the web search agent to only use data fetching tools and ignore analysis tools

Prompt instructions are probabilistic and the agent is already misusing tools. The architectural fix is to remove the tools it should not have, providing deterministic enforcement through scoped tool access.

D Merge the web search and synthesis agents into a single agent since they share some tools

Merging agents violates the principle of specialised subagents. The solution is to separate tools by role, not to combine agents. Merging would create an even larger tool set on a single agent.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access (4-5 tools))

Anthropic: Claude Agent SDK Overview ↗

Q48 · D4 · 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS · q-4-3-001 · Structured Data Extraction Pipeline · INCORRECT

Your extraction system uses tool\_use with a strict JSON schema. All fields are marked as required. Testers report that the model invents plausible-looking dates and amounts when processing documents that lack this information. What is the best fix?

A Add an instruction telling the model not to hallucinate

Vague instructions do not override the schema constraint. Required fields pressure the model to produce values.

**C** Make fields optional/nullable when source documents may not contain the information

Optional/nullable fields allow the model to return null instead of fabricating values. This directly prevents fabrication for missing information. **(correct answer)**

**D** Add a validation step that checks all values against the source document

Post-hoc validation is useful but does not address the root cause. Making fields optional prevents fabrication at the schema level. **(your answer)**

**B** Switch from tool\_use to prompt-based JSON extraction

This moves backwards in reliability. Prompt-based JSON introduces syntax errors without solving the fabrication problem.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable / optional fields)

Q49 · D4 · 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING · q-4-3-002 · Structured Data Extraction Pipeline **INCORRECT**

Your legal document extraction pipeline uses tool\_use with tool\_choice set to 'auto'. The pipeline processes contracts and sometimes receives plain text analysis instead of the structured JSON extraction you need. The team suggests switching to prompt-based JSON with explicit formatting instructions. What is the correct approach?

**D** Add stronger instructions in the system prompt telling the model to always use the extraction tool and never respond with plain text

Instructions alone cannot guarantee tool use when tool\_choice is 'auto'. The API-level tool\_choice parameter is the correct mechanism for enforcing structured output, not prompt instructions.

**A** Switch to prompt-based JSON as suggested, since the model clearly prefers text responses for these documents

Prompt-based JSON is less reliable than tool\_use and can produce malformed output. The issue is tool\_choice configuration, not the tool\_use mechanism itself.

**B** Keep tool\_use but switch tool\_choice from 'auto' to 'any' to guarantee the model always returns a structured tool call

While 'any' guarantees a tool call, it lets the model choose which tool. If you have multiple tools defined, the model might call the wrong one. For a single extraction tool, this works, but the most precise solution is to force the specific extraction tool. **(your answer)**

**C** Keep tool\_use and set tool\_choice to force the specific extraction tool by name, guaranteeing structured output on every request.

Forcing a specific tool with tool\_choice (type: 'tool', name: 'extract\_contract') guarantees the model calls that exact tool every time, eliminating both text-only responses and wrong-tool selection. This is the most reliable approach for guaranteed structured output. **(correct answer)**

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (tool\_choice forcing)

Anthropic: Tool Use Documentation ↗

Q50 · D1 1.6 TASK-DECOMPOSITION / DELEGATION q-1-6-008 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code **CORRECT**

A coordinator agent receives a request to extract a payment processing module from the monolith into a standalone microservice. The task involves creating new API endpoints, migrating database tables, updating 40+ call sites, and writing integration tests. A junior developer suggests the coordinator should handle the entire task itself to avoid subagent communication overhead. Why is this approach wrong?

**D** The coordinator's API rate limits would be exceeded when processing 40+ files

API rate limits are a practical concern but not the architectural reason to delegate. The root issue is attention dilution — quality degrades when one agent processes too many items in a single context, regardless of rate limits.

**C** The coordinator cannot access file system tools, so it physically cannot make code changes

There is no inherent restriction preventing a coordinator from having file system tools. The issue is architectural — a single agent processing too many files suffers from attention dilution, not tool access limitations.

**B** Loading 40+ files into one context dilutes attention — the coordinator should delegate to scoped subagents instead

Large, multi-file tasks suffer from attention dilution when processed in a single context. Delegating to specialist subagents ensures each agent works with focused context. The coordinator should handle task decomposition and context injection, delegating implementation to specialists.

**A** The coordinator should never write code — it should only route tasks

The coordinator can handle simple, well-scoped tasks directly. The issue here is not a blanket rule against the coordinator writing code, but that this specific task is too large and cross-cutting for a single agent to handle effectively.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Attention dilution)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q51 · D3 3.4 PLAN-MODE-EXECUTION / EXPLORE-SUBAGENT q-3-4-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code **CORRECT**

A developer asks Claude Code to 'explain the order processing pipeline' before refactoring it. The pipeline spans 15 classes across 4 packages with deep inheritance and callback chains. Claude Code reads 3 files and produces a superficial summary that misses the callback chain entirely. What approach would produce a more thorough analysis?

**A** Re-prompt Claude Code with more specific instructions listing all 15 classes to read

This assumes the developer already knows which 15 classes are involved. If the developer knew the full scope, they would not need Claude Code's help understanding the pipeline. The issue is that Claude Code's initial exploration was too shallow.

**B** Use an Explore subagent for verbose discovery across the pipeline's classes and callback chains

The Explore subagent is purpose-built for verbose, thorough codebase discovery. It systematically follows references, traces inheritance, and maps control flow across files — exactly what is needed for a complex event-driven pipeline. Regular Claude Code defaults to efficient, minimal reads; the Explore subagent prioritises comprehensive discovery.

**D** Run multiple Claude Code sessions, each analysing a different package, then manually combine the results

Splitting analysis across sessions loses the cross-package context needed to trace the callback chains and inheritance hierarchies. The pipeline's complexity comes from how the 4 packages interact, which requires a single comprehensive analysis.

**C** Switch to plan mode and ask Claude Code to create a plan for understanding the pipeline

Plan mode can safely explore a codebase before you commit to changes, but it runs in the main conversation and drives toward a change plan. Here the developer needs thorough, verbose discovery of current behaviour, which the Explore subagent isolates and summarises without exhausting the main context, so the Explore subagent is the better fit.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Explore subagent)

Q52 · D3 3.2 SLASH-COMMANDS-SKILLS / CONTEXT-FORK q-3-2-015 · Technical Documentation Maintenance System

CORRECT

A documentation team creates a /generate-api-docs skill that reads source code files and produces Markdown API reference pages. The skill generates verbose output (200+ lines per endpoint) and should be available to every team member who clones the repository. How should the skill be configured?

**B** Create a SKILL.md in .claude/skills/ with context: fork, so it is shared via git and isolates the verbose output

.claude/skills/ is project-scoped and version-controlled, making it available to every developer who clones the repository. context: fork isolates the verbose 200+ line output per endpoint into a forked context, preventing it from consuming the main conversation's context budget.

**D** Create a SKILL.md in .claude/skills/ without any frontmatter, relying on the team to manually manage context overflow

Without context: fork, the verbose output (200+ lines per endpoint across many endpoints) would flood the main conversation context. The context: fork frontmatter is essential for isolating bulky output.

**C** Add the documentation generation instructions to the root CLAUDE.md so they load automatically in every session

CLAUDE.md is for always-loaded standards, not for on-demand command definitions. Loading verbose generation instructions in every session wastes context tokens when the team is doing non-documentation work.

**A** Create a SKILL.md in ~/.claude/skills/ with context: fork frontmatter, and instruct each team member to copy it locally

~/.claude/skills/ is user-scoped and not version-controlled. Requiring manual copying defeats the purpose of team-shared configuration and creates maintenance drift.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (context: fork)

Claude Code: Skills ↗

Q53 · D5 · 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING · q-5-5-009 · Structured Data Extraction Pipeline

CORRECT

Your extraction pipeline reports 97% aggregate accuracy, and you want to automate high-confidence extractions. Which safeguards should be in place before you reduce human review? (Select 3)

**A** Calibrate field-level confidence thresholds against a labelled validation set.

Confidence scores only route review attention correctly once they are calibrated on labelled data.

**B** Rely on the aggregate accuracy figure once it has held steady for a full quarter.

Stability of the aggregate number does not reveal per-segment weaknesses, which is the risk the guide highlights.

**D** Keep stratified random samples of high-confidence extractions under review to measure error rates and catch novel patterns.

Stratified sampling is the guide's mechanism for ongoing measurement after automation.

**E** Expand the review team until every extraction can be checked by hand.

Reviewing everything ignores the point of calibration: routing limited reviewer capacity to low-confidence and ambiguous cases.

**C** Verify accuracy separately by document type and field segment, not just in aggregate.

A strong aggregate figure can mask poor performance on specific document types or fields.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)

Q54 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / CONTEXT-PASSING q-1-3-015 · Multi-Agent Research System CORRECT

A coordinator spawns a research subagent to investigate market trends. The subagent returns detailed findings. The coordinator then needs to pass these findings to a synthesis subagent for report writing. However, the synthesis subagent produces a report that contradicts the research findings. What is the most likely cause?

D The two subagents are using different model versions with different training data

Model version differences would not cause contradictions with explicitly provided research data. The issue is that the research data is not being fully passed to the synthesis agent, not that the agents have different base knowledge.

C The synthesis subagent has its own web search tool and keeps pulling contradictory information from the open web instead of the research it was handed.

A synthesis subagent should not have web search tools; that violates scoped tool access. The real issue is that the full research output is not being passed to it, not that it is searching the web.

B The coordinator passes a summary instead of the full structured research output, so the synthesis agent fills gaps from its training data.

Subagents do not share memory. The coordinator must explicitly pass the complete research output to the synthesis agent. If the coordinator only passes a summary, the synthesis agent has no access to the detailed findings and will rely on its training data to fill gaps, potentially contradicting the actual research.

A The synthesis subagent has a conflicting system prompt that overrides the research data

While possible, conflicting system prompts would produce consistently wrong outputs across all requests. The more common cause is that the coordinator is not passing the full research output, so the synthesis agent fills gaps with its own training data.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Q55 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-005 · Multi-Agent Research System CORRECT

A research coordinator spawns 4 subagents to investigate different aspects of a market analysis. Agent A fails with an API error, Agent B returns partial results (3 of 5 requested data points), and Agents C and D return complete results. The final report is due in 2 hours. What should the coordinator do?

C Retry Agent A, incorporate Agent B's partial results, flag B's missing data points as gaps, and synthesise from all available results.

The coordinator should maximise data utilisation: retry the failed agent (A), incorporate partial results from B while flagging the 2 missing data points as known gaps, and synthesise all available results from A (after retry), B (partial), C,



and D. Explicit gap annotations ensure the report is transparent about its completeness.

**B** Proceed with only Agents C and D's results and produce the report without the data from Agents A and B

This unnecessarily discards Agent B's partial results and abandons Agent A's scope entirely. A thorough coordinator should attempt recovery while incorporating all available data.

**D** Terminate all agents and restart the entire investigation from scratch with a simplified scope

Restarting discards the complete results from Agents C and D and the partial results from Agent B. This wastes time and compute when most of the investigation has already succeeded.

**A** Retry Agent A, discard Agent B's partial results as unreliable, and wait for both agents to return fully complete data before writing the report.

Discarding Agent B's partial results wastes valid data, and waiting for full completion ignores the two-hour deadline. The coordinator should use what is available while attempting to fill gaps.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.7: Session State and Resumption (Recovering from partial results)

Q56 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-005 · Multi-Agent Research System CORRECT

A synthesis agent receives findings from three subagents: solar investment grew 15% (Bloomberg, January 2024), solar investment grew 22% (IEA, March 2024), and cost-per-watt data. The synthesis agent currently picks the IEA figure because it is more recent. What is the correct approach?

**C** Present both figures with source attribution and dates, noting what may explain the difference.

Preserving both data points with provenance — source, date, and potential explanations for the difference — lets the consumer interpret the data correctly. Different publication dates and methodologies explain the discrepancy without requiring the synthesis agent to judge which is 'right'.

**A** Always use the most authoritative source (IEA as an international agency) and discard the Bloomberg figure.

Discarding a credible source based on perceived authority silently removes valid data. Both sources are credible, and the difference may reflect different methodologies, scopes, or time periods.

**B** Average the two values (18.5%) and cite both sources to present a balanced view.

Averaging conflicting statistics is mathematically misleading. The two figures may measure different scopes (e.g., different regions or investment types), making an average meaningless. The difference needs to be preserved, not merged.

**D** Flag the conflicting figures and pause report generation until a human researcher resolves the discrepancy.

Pausing for human resolution is unnecessary when the difference can be explained by temporal or methodological factors. Proper annotation with provenance resolves the ambiguity without blocking the workflow.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q57 · D2 · 2.4 MCP-SERVER-INTEGRATION / DESCRIPTIONS · q-2-4-006 INCORRECT

An MCP server exposes a 'query\_database' tool for Snowflake. Agents ignore it and run SQL via the built-in Bash tool against the Snowflake CLI, even though the MCP tool returns structured results with column types and pagination that Bash does not. What is the most likely cause and fix?

**C** Disable the Bash tool entirely so the agent is forced to use the MCP tool for all operations.

Disabling Bash removes a critical general-purpose tool needed for many operations beyond database queries. The fix should make the MCP tool more attractive for its specific use case, not remove other tools.

**A** The MCP server is not properly connected. Restart the MCP server and verify the connection with a test query.

If the server were disconnected, the tool would not appear in the available tools list at all. The agent is choosing not to use it, which is a description problem, not a connectivity problem.

**D** Add a system prompt instruction telling the agent to always use query\_database instead of Bash for SQL queries.

Whilst a system prompt instruction could work as a short-term fix, it is brittle and does not scale. The root cause is the sparse description. Enhancing the description is the sustainable fix that works across all agents and contexts. (your answer)

**B** Enhance the sparse description to spell out the tool's structured output and pagination advantages over Bash.

When MCP tool descriptions are sparse, agents default to the familiar Bash tool. Enhancing the description to explain the structured results, column types, and pagination gives the agent enough to prefer the MCP tool for database queries. (correct answer)

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

Q58 · D3 · 3.6 CICD-INTEGRATION / CLAUDE-MD-CRITERIA · q-3-6-006 · INCORRECT  
Large-Scale Codebase Refactoring with Multi-Agent Claude Code

The team's CLAUDE.md specifies that all new microservices must include integration tests that verify API contract compliance and database migration rollback. A developer notices that Claude Code sometimes generates unit tests that mock the database instead of writing the required integration tests. What should the team add to their configuration?

**C** A separate testing subagent that reviews generated tests and rewrites any that use mocks

A review subagent adds complexity and is still probabilistic. Clear specifications in CLAUDE.md solve the issue at the source by telling Claude Code exactly what to generate, rather than correcting after generation.

**A** A PostToolUse hook that rejects any test file containing mock annotations

Rejecting all mocks is too aggressive — unit tests with mocks are still valuable for other purposes. The issue is that integration tests are missing, not that unit tests exist. The configuration should specify what is required, not ban useful testing patterns.

**B** Tighten the CLAUDE.md test rule with criteria like real DB connections and contract assertions

CLAUDE.md is the correct location for persistent test standards. Specifying concrete criteria (real database connections, API contract assertions) with examples of expected structure and fixtures gives Claude Code an unambiguous target. This addresses the root cause: the existing instructions were not specific enough about what 'integration tests' means in this codebase. **(correct answer)**

**D** A .claude/rules/ file with paths: ["\*\*/\*Test.java"] that instructs Claude Code to never use mocks

Banning mocks from all test files is overly broad — mocks are appropriate in many unit tests. The requirement is specifically that integration tests with real connections must be included, not that mocks must be eliminated. **(your answer)**

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (CLAUDE.md for CI context)

Q59 · D1 1.2 ORCHESTRATION-PATTERNS / DECOMPOSITION q-1-2-001 · Multi-Agent Research System **INCORRECT**

A multi-agent research system produces a report on 'renewable energy technologies' that only covers solar and wind, missing geothermal, tidal, biomass, and nuclear fusion. Each subagent produced thorough coverage of its assigned topic. Where is the root cause?

**A** The web search subagent's search queries were too narrow

The web search subagent researched what it was assigned. The issue is what it was assigned, not how it searched.

**D** The document analysis subagent lacked access to sources on geothermal, tidal, biomass, and fusion, so those categories never entered the report.

Source availability is not the root cause. The coordinator never asked any agent to research geothermal, tidal, biomass, or fusion, so no fix to source access would help. **(your answer)**

**B** The synthesis subagent failed to identify gaps in the research

The synthesis agent can only work with the research it receives. It cannot synthesise topics that were never researched.

**C** The coordinator's task decomposition assigned only solar and wind, omitting the other renewable categories.

The coordinator is responsible for decomposing the broad topic into subtopics. If it only assigns solar and wind, no downstream agent can cover the missing categories. Trace failures to their origin. **(correct answer)**

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Narrow decomposition failure)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q60 · D5 · 5.5 HUMAN-REVIEW-CALIBRATION / PER-FIELD-CALIBRATION · q-5-5-005 · Structured Data Extraction Pipeline **CORRECT**

A contract extraction system reports 96% overall accuracy. The team plans to auto-approve all extractions where model confidence exceeds 90%. A pilot reveals party name extraction achieves 99% accuracy but indemnification clause extraction only 71%, even though the model reports high confidence on both. What should they implement before automating?

**B** Calibrate confidence thresholds per field type using labelled validation sets, and implement stratified sampling to continuously monitor accuracy by document type and field segment.

Field-level confidence calibration using ground truth data exposes the discrepancy between reported confidence and actual accuracy. Stratified sampling provides ongoing monitoring to detect novel error patterns. Together, these ensure automation is only applied where validated accuracy justifies it.

**A** Raise the automation confidence threshold from 90% to 99% to ensure only the most reliable extractions are automated.

Raising the threshold does not fix the calibration problem. The model reports high confidence even for indemnification clauses where it achieves only 71% accuracy. Raw confidence scores are unreliable without calibration.

**D** Train a separate classification model to predict extraction accuracy before deciding whether to automate each extraction.

A separate model adds infrastructure complexity when the solution is to calibrate the existing confidence scores against labelled data. Calibration directly addresses the unreliable confidence outputs without requiring additional ML infrastructure.

**C** Exclude indemnification clauses from automation entirely and continue automating all other field types at the 90% threshold.

This addresses the known problem but does not detect future calibration issues with other field types. Without systematic calibration and monitoring, other fields with hidden accuracy problems will be automated incorrectly.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)  
Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

SUPPORT

Found this useful?

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.

☞ BUY ME A COFFEE